

User Stories — Frictionless Guest Checkout

The stories below cover the in-scope behaviour of the Epic. Each carries a brief acceptance summary; full Gherkin scenarios are in acceptance-criteria.md.

US-01 — Browse products

As a visitor I want to see a catalogue of available products with name, price, and a short description **So that** I can decide what to buy.

Acceptance summary: home page renders a grid of products from the catalogue with image, title, price (USD), and a single “Add to cart” action per product.

US-02 — Add items to cart

As a visitor I want to add items to a cart from the catalogue **So that** I can purchase them later in a single transaction.

Acceptance summary: clicking “Add to cart” places one unit of that product into the cart and updates the cart count in the header. Adding the same product again increases the quantity.

US-03 — Review and adjust the cart

As a visitor I want to view, remove, and change quantities in my cart **So that** I can finalise what I’m buying before paying.

Acceptance summary: cart page lists every item with quantity controls (+/-), a “Remove” action, line totals, a subtotal, and a “Proceed to checkout” CTA. Quantity 0 removes the item.

US-04 — Check out as a guest

As a visitor without an account I want to complete a purchase without creating an account **So that** I’m not blocked by signup friction.

Acceptance summary: the checkout form is reachable directly from the cart and accepts billing details, an address, and a card without any login or signup step.

US-05 — Receive an order confirmation email

As a guest I want to receive an email confirmation after a successful payment **So that I** have proof of purchase and the order number.

Acceptance summary: on successful payment, the system sends an email via SendGrid containing the order number, items, total, and the address the order will ship to. Welcome and onboarding emails go to all users on first checkout.

US-06 — Address autocomplete

As a guest entering my shipping address I want to see address suggestions as I type **So that** entry is fast and accurate.

Acceptance summary: as the user types in the address field, suggestions appear from Google Places. Selecting one fills the city, postcode, and country fields.

US-07 — Pay by card via Stripe

As a guest I want to pay with my credit or debit card **So that** I can complete the purchase.

Acceptance summary: Stripe Elements is embedded in the checkout form. On submit, the client creates a payment intent against `/api/create-payment-intent`, confirms with the user's card, and on success redirects to the confirmation page.

US-08 — Card number validation

As a guest I want the form to catch obvious card number mistakes before I submit **So that** I don't waste a round trip to Stripe.

Acceptance summary: the form runs a Luhn check on the card number client-side and surfaces an inline error when the check fails. The form continues to allow saved cards if the user has any on file.

US-09 — Order confirmation page

As a guest I want to see a confirmation page after payment **So that** I know my order went through and can capture the order number.

Acceptance summary: on success, the user lands on `/confirmation` with the order number visible, a summary of what was bought, the total charged, and a single CTA to continue shopping.

US-10 — Resilience under transient payment failures

As a product team **We want** the system to retry transient Stripe failures automatically **So that** users aren't punished for blips in the payment network.

Acceptance summary: the server retries Stripe calls up to 3 times on `network_error` and `gateway_timeout` with exponential backoff. Permanent failures (`card_declined`, etc.) are surfaced immediately without retry.

US-11 — Per-IP rate limiting

As a platform owner **I want** payment endpoints rate-limited per IP **So that** abuse and brute-force attempts are blunted.

Acceptance summary: `/api/create-payment-intent` rejects requests exceeding 10 per minute per IP with HTTP 429.

US-12 — Downstream notification of paid orders

As a fulfilment system owner **I want** a `payment.completed` event published for each paid order **So that** downstream systems can act on it asynchronously.

Acceptance summary: on successful payment confirmation, the server publishes an event to the orders Kafka topic containing the order ID, total, currency, and a list of items.